Powering Up Your Types with Annotated

Vlad Fedoriuk

Who am I

5+ years of working with Python · Software Engineer @ **ActiveCampaign**

Two distinct usage patterns

SEMANTIC TYPES

name what data *is*

FRAMEWORK METADATA

instructions for tools

Use case: semantic types

Same type — different meaning.

```
from typing import Annotated
from sqlalchemy import Engine

ReaderEngine = Annotated[Engine, "Reader"]
WriterEngine = Annotated[Engine, "Writer"]
```

Use case: framework metadata

Attach instructions for tools — Pydantic, FastAPI, SQLAlchemy.

```python
from typing import Annotated
from pydantic import BaseModel, Field
from fastapi import FastAPI, Query


class SearchResult(BaseModel):
    total: Annotated[int, Field(ge=0)]


app = FastAPI()


@app.get("/search/")
def search(
    q: Annotated[str, Query(min_length=3)],
): ...
```

Annotated 101

`__origin__` and `__metadata__`

`__origin__` is the base type; `get_origin()` returns `Annotated`.

```
from typing import Annotated, get_origin
from annotated_types import MinLen, MaxLen

Name = Annotated[str, MinLen(1), MaxLen(100)]

Name.__origin__  # str
Name.__metadata__  # (MinLen(1), MaxLen(100))

get_origin(Name)  # Annotated  ← not str!
```

Nesting flattens

Nested `Annotated` merges into one tuple — inner metadata first.

```
from typing import Annotated
from annotated_types import MinLen, MaxLen

Inner = Annotated[int, MinLen(1)]
Outer = Annotated[Inner, MaxLen(100)]  # nested Annotated flattens

Outer.__metadata__  # (MinLen(1), MaxLen(100))  ← inner metadata first
Outer == Annotated[int, MinLen(1), MaxLen(100)]  # True
```

Aliases block flattening

PEP 695 `type` aliases are *lazy* — outer `Annotated` cannot see inside until evaluated.

```python
from typing import Annotated
from annotated_types import MinLen, MaxLen

type Inner = Annotated[int, MinLen(1)]

Annotated[Inner, MaxLen(100)].__metadata__
# (MaxLen(100),)  ← MinLen(1) is hidden inside Inner

Annotated[Inner, MaxLen(100)] == Annotated[int, MinLen(1), MaxLen(100)]
# False  ← alias breaks equality too
```

Annotated is not a type

Special form — not assignable to `type[T]`.

```python
from typing import Annotated

type(Annotated[int, "x"])
# <class 'typing._AnnotatedAlias'>

isinstance(Annotated[int, "x"], type)
# False
```

The problem

Real collision — `Annotated` passed as `type[T]`.

```python
def register_value(svc_type: type[T], value: T) -> None: ...


register_value(Annotated[int, "my_int"], 42)
# ↑ type error in mypy, pyright, ty, pyrefly
```

The fix — TypeForm

PEP 747 — accepted for Python 3.15. Backport: `typing_extensions.TypeForm`.

```python
# After — PEP 747, Python 3.15+
from typing import TypeForm  # or: from typing_extensions import TypeForm


def register_value(svc_type: TypeForm[T], value: T) -> None: ...


register_value(Annotated[int, "x"], 42)  # ✓ OK
```

svcs 26.1.0

Released July 13, 2026. Ships `TypeForm` support out of the box.

```
@overload
async def aget(self, svc_type: TypeForm[T1], /) -> T1: ...
```

Consuming metadata at runtime

Example classes

```python
from decimal import Decimal
from typing import Annotated

from annotated_types import Ge, Gt, Le, MaxLen, MinLen

Name = Annotated[str, MinLen(1), MaxLen(100)]
Price = Annotated[Decimal, Gt(0)]
Percentage = Annotated[Decimal, Gt(0), Le(100)]


class HasName:
    name: Name


class Product(HasName):
    price: Price
    discount: Percentage | None
```

Read annotations with the API

New in Python 3.14 — supersedes `inspect.get_annotations()`. Low-level tool; `typing.get_type_hints()` builds on top.

```
import annotationlib

# Preferred — own annotations only, never traverses MRO
annotationlib.get_annotations(HasName)
# {'name': Annotated[str, MinLen(1), MaxLen(100)]}

# Avoid __annotations__ directly — it has edge cases on older Pythons
HasName.__annotations__
# {'name': Annotated[str, MinLen(1), MaxLen(100)]}

# Use the API instead:
#   annotationlib.get_annotations()  Python 3.14+
#   inspect.get_annotations()        Python 3.10+
```

annotationlib.get_annotations

Three formats — evaluated objects, source strings, or forward-reference proxies.

```
from annotationlib import Format, ForwardRef, get_annotations

get_annotations(HasName, format=Format.VALUE)
# {'name': Annotated[str, MinLen(1), MaxLen(100)]}   ← evaluated

get_annotations(HasName, format=Format.STRING)
# {'name': 'Name'}                                   ← source text, no eval


class Early:
    sibling: Pending  # not yet defined


get_annotations(Early, format=Format.FORWARDREF)
# {'sibling': ForwardRef('Pending')}                 ← proxy, no NameError
```

typing.get_type_hints

Merges MRO, resolves forward refs — strips `Annotated` unless `include_extras=True`.

```
from typing import get_type_hints

get_type_hints(Product)
# {'name': str, 'price': Decimal, ...}  ← stripped, full MRO

get_type_hints(Product, include_extras=True)
# {'name': Name, 'price': Price, ...}  ← Annotated preserved
```

annotated-types

Shared constraint vocabulary.

`Gt` `Ge` `Lt` `Le` `Interval` `MultipleOf` `MinLen` `MaxLen` `Len` `Timezone` `Predicate`

`IsFinite` `IsNotNan`

BaseMetadata + GroupedMetadata

```
@dataclass(frozen=True)
class BaseMetadata:
    """Base for simple constraints: Gt, Le, MinLen, …"""


class GroupedMetadata(Protocol):
    def __iter__(self) -> Iterator[BaseMetadata]: ...

    # Interval → Gt + Lt, Len → MinLen + MaxLen
```

The get_metadata idiom

Suggested by `annotated-types` — walk metadata that implements `BaseMetadata` or `GroupedMetadata`.

```
from typing import Annotated, get_args, get_origin
from annotated_types import BaseMetadata, GroupedMetadata


def get_metadata(tp):
    assert get_origin(tp) is Annotated
    for arg in get_args(tp)[1:]:  # [0] is the base type
        if isinstance(arg, BaseMetadata):
            yield arg
        elif isinstance(arg, GroupedMetadata):
            yield from arg  # e.g. Interval → Gt + Lt
```

Toy ~~Dependency Injection~~
Auto-wiring

Auto-wiring with Annotated

Metadata-driven autowiring — `Depends` as `BaseMetadata`.

```python
from dataclasses import dataclass
from annotated_types import BaseMetadata


@dataclass
class Depends(BaseMetadata):
    dependency: Callable


# usage — metadata on the parameter annotation
def current_user(env: Annotated[str, Depends(settings)]) -> str: ...
```

Filter constraints for Depends

`get_metadata` yields `BaseMetadata` — `next()` picks the first `Depends`.

```
def resolve_dependencies(fn):
    injected = {}
    for name, hint in get_type_hints(fn, include_extras=True).items():
        if get_origin(hint) is not Annotated:
            continue
        dep = next((m for m in get_metadata(hint) if isinstance(m, Depends)), None)
        if dep is not None:
            injected[name] = dep.dependency(**resolve_dependencies(dep.dependency))
    return injected
```

Call with injected deps

Resolve dependency graph, call with `**deps`.

```
def config() -> str:
    return "prod"

def db_url(env: Annotated[str, Depends(config)]) -> str:
    return f"db://{env}"

def endpoint(url: Annotated[str, Depends(db_url)]) -> str:
    return f"connected to {url}"


deps = resolve_dependencies(endpoint)
endpoint(**deps)  # "connected to db://prod"
```

What the toy leaves out

DEDUPLICATION & CACHING

same dep resolved once per scope; parsed signatures & type hints cached across calls

ASYNC & GENERATORS

`async def` deps, `yield`-based teardown

SCOPING

per-call isolated state — no leaks across concurrent requests

OVERRIDING

swap deps in tests without touching call sites

The DI ecosystem

FastDepends powers FastStream · uncalled-for powers FastMCP

FastDepends svcs di Lagom diwire anydi Picodi andi uncalled-for

python-dependency-injector Injector Dishka Inject Kink Punq Wireup That Depends

Rodi python-injection injectable Opyoid myfy Modern DI Injex ididi injection

Fresh Bakery engin Clean IoC Overlay

Let’s build something

Room Reservation

```
@dataclass
class Reservation:
    room_id: str
    guest_count: int
    starts_at: datetime.date
    ends_at: datetime.date
    rate: Decimal

@dataclass
class Room:
    room_id: str
    capacity: int
    reservations: list[Reservation] = field(default_factory=list)

    def add_reservation(self, reservation: Reservation) -> None:
        if reservation.guest_count > self.capacity:
            raise ValueError("Exceeds room capacity")
        ...
```

Make your types mean something

Name what data is — and what's valid.

```python
# Step 2: Swap to rich semantic types
RoomId = Annotated[str, MinLen(1), MaxLen(20)]
GuestCount = Annotated[int, Ge(1), Le(10)]
RoomRate = Annotated[Amount, Gt(0)]
TimestampTz = Annotated[datetime.datetime, Timezone(...)]


@dataclass
class Reservation:
    room_id: RoomId
    guest_count: GuestCount
    rate: RoomRate
    created_at: TimestampTz
```

Polyfactory: almost zero config

`Ge`, `Le`, `Gt`, `MinLen`, `MaxLen` — resolved automatically.

```
class ReservationFactory(DataclassFactory[Reservation]):
    __model__ = Reservation

    # PostGenerated: ends_at must follow starts_at
    ends_at = PostGenerated(
        lambda name, values, *args, **kwargs: (
            values["starts_at"] + datetime.timedelta(days=random.randint(1, 14))
        )
    )
    # Timezone(...) bug: polyfactory passes ... as tzinfo
    created_at = Use(
        faker.date_time_between, start_date="-30d", end_date="now", tzinfo=UTC
    )
```

Using factories in tests

```
def test_duplicate_reservation_is_rejected() -> None:
    reservation = ReservationFactory.build(room_id="101")
    room = RoomFactory.build(
        room_id="101",
        reservations=[reservation],
    )

    with pytest.raises(ValueError, match="Reservation overlaps existing reservation"):
        room.add_reservation(reservation)
```

Hypothesis: property-based testing

`st.builds` builds objects, `st.from_type` reads `Annotated` constraints.

```python
@dataclass
class StayRequest:
    guest_count: GuestCount
    room_id: RoomId
    starts_at: date
    ends_at: date
```

```python
@given(
    req=st.builds(StayRequest),
    discount=st.from_type(Percentage),
    tax=st.from_type(TaxRate),
)
def test_stay_total(req, discount, tax):
    assume(req.starts_at <= req.ends_at)
    r = Reservation(
        room_id=req.room_id,
        guest_count=req.guest_count,
        starts_at=req.starts_at,
        ends_at=req.ends_at,
        rate=Decimal("100"),
    )
    assert calculate_stay_total(r, discount, tax) >= 0
```

SQLAlchemy: reusable column blueprints

ORM config in the alias — table-specific constraints on the field.

```python
# Step 3: Use the aliases in a model
Identity = Annotated[
    int,
    Gt(0),
    mapped_column(sa.Integer(), sa.Identity(always=False), primary_key=True),
]


@registry.mapped_as_dataclass
class Room:
    __tablename__ = "rooms"

    id: Mapped[Identity] = mapped_column(init=False)
    room_id: Mapped[RoomId] = mapped_column(unique=True, index=True)
    rate: Mapped[RoomRateColumn]
    created_at: Mapped[TimestampTz]
```

Generating migrations

Alembic reads the models — you write nothing.

```
alembic revision --autogenerate -m "Initial migrations"
alembic upgrade heads
```

```
op.create_table(
    "rooms",
    sa.Column("id", sa.Integer(), sa.Identity(), nullable=False),
    sa.Column("room_id", sa.String(20), nullable=False),
    sa.Column("capacity", sa.Integer(), nullable=False),
    sa.PrimaryKeyConstraint("id"),
)
op.create_index("ix_rooms_room_id", "rooms", ["room_id"], unique=True)
# ...
```

Pydantic & FastAPI: composable schemas

Same types for validation and OpenAPI out of the box.

```
class CreateReservationSchema(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)

    room_id: Annotated[RoomId, Field(title="Room ID")]
    guest_count: GuestCount
    starts_at: StayDate
    ends_at: StayDate
    rate: RoomRate
```

Validation & serialization as metadata

`BeforeValidator` `AfterValidator` `PlainValidator` `WrapValidator` `WithJsonSchema`

`PlainSerializer`

```python
# Step 2: Overlay validation, serialization & usage
Amount = Annotated[
    Decimal,
    Gt(0),
    WrapValidator(lambda v, handler: handler(v).quantize(Decimal("0.01"))),
    PlainSerializer(lambda v: float(v), return_type=float, when_used="always"),
]

ta = TypeAdapter(Amount)
assert ta.validate_python(Decimal("10.123")) == Decimal("10.12")
assert ta.validate_python(10.456) == Decimal("10.46")
assert ta.dump_json(Decimal("10.12")) == b"10.12"
```

Fits together with FastAPI

```python
@app.get("/rooms/{room_id}")
async def get_room(
    room_id: Annotated[RoomId, Path(title="Room ID")],
    services: svcs.fastapi.DepContainer,  # Annotated[Container, Depends(container)]
) -> RoomResponse: ...


@app.post("/reservations/", status_code=status.HTTP_201_CREATED)
async def create_reservation(
    data: CreateReservationSchema,
    services: svcs.fastapi.DepContainer,
) -> ReservationSchema:
    service = await services.aget(ReservationService)
    reservation = await service.place(data)
    return ReservationSchema.model_validate(reservation, from_attributes=True)
```

Nest. Overlay. Compose.

Orthogonal metadata

```python
RoomId = Annotated[
    str,
    MinLen(1), MaxLen(20),              # domain constraint
    BeforeValidator(trim_str),          # validation at edges
    mapped_column(sa.String(20)),       # persistence
]
```

Domain stays central

Stack orthogonal concerns — neither DB nor API has the last word.

Types as first-class citizens

Types define what data *is* — not just what the type checker accepts.

Ecosystem converges

`annotated-types` contracts — shared vocabulary, free composition.

Why reinvent the wheel?

The Django way

```
class Room(models.Model):
    room_id = models.CharField(max_length=20, unique=True, db_index=True)
    capacity = models.PositiveSmallIntegerField(
        validators=[MinValueValidator(1), MaxValueValidator(10)]
    )


class RoomSerializer(serializers.ModelSerializer):
    class Meta:
        model = Room
        fields = ["id", "room_id", "capacity"]
```

The SQLModel way

```python
class RoomBase(SQLModel):
    room_id: RoomId = Field(index=True, unique=True, max_length=20)
    capacity: GuestCount = Field(ge=1, le=10)


class Room(RoomBase, table=True):
    id: int | None = Field(default=None, primary_key=True)
```

Structural pressure

- **DB and API pull on each other**

  Change a column, ripple through the endpoint. One has the upper hand.

- **Great when stable**

  CRUD, low volatility, no backward-compatibility dance — these tools shine.

- **Do repeat yourself**

  Duplicate first — refactor once understanding emerges.

Design pressure — https://hynek.me/talks/design-pressure/

One more layer

Documentation: annotated-doc

`Doc` metadata inline in type signatures.

```
from annotated_doc import Doc

FIRST_ORDER_DISCOUNT: Annotated[
    Percentage,
    Doc("Discount percentage applied to the first reservation made by a guest"),
] = Decimal(15)
```

MkDocs / Zensical

```yaml
# mkdocs.yml
plugins:
  - mkdocstrings:
      handlers:
        python:
          options:
            extensions:
              - griffe_typingdoc
            import_paths: [...]
```

Render `Doc` annotations automatically — see zensical docs.

Thank you!

questions?